JAWAHARLAL COLLEGE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF AERONAUTICAL ENGINEERING

Lakkidi, Palakkad, Kerala — 679301

INSTANT FLIGHT TRAJECTORY PREDICTOR

A Physics-Informed Machine Learning Surrogate for Ballistic Flight Dynamics with Aerodynamic Drag

A Project Report submitted in partial fulfillment of the academic requirements for the course

COMPUTATIONAL FLIGHT MECHANICS & DATA SYSTEMS (AE401)

SUBMITTED BY TEAM NINE:

Student Name	Roll Number	Department
Vignesh VS	JCEAO057	Aeronautical Engineering
Vishnu K	JCEAO058	Aeronautical Engineering
Vivek Raj	JCEAO059	Aeronautical Engineering
Yuktha Patikkal	JCEAO060	Aeronautical Engineering
Muhammed Nishad	JCEAO061	Aeronautical Engineering
Muhammed Navas	JCEAO062	Aeronautical Engineering

ABSTRACT

Numerical integration of nonlinear flight equations of motion—such as classical fourth-order Runge-Kutta (RK45) routines for ballistic projectiles subject to quadratic aerodynamic drag—incurs non-negligible computational overhead, severely limiting its utility in real-time guidance, swarm trajectory prediction, and low-power embedded avionics. This project presents the ***Instant Flight Trajectory Predictor****, a full-stack Machine Learning surrogate framework capable of predicting five critical flight parameters (Range, Max Altitude, Flight Time, Apex X, and Impact Speed) in under 0.27 ms, representing a 4x to 21x speedup over numerical ODE solvers. Trained on a 5,000-sample flight dataset spanning velocities from 20 to 300 m/s, launch angles from 5° to 85°, and masses from 0.5 to 50 kg, our proposed physics-informed polynomial ridge regression model achieves $R^2 > 0.999$ with a Mean Absolute Error of 22.3 m across multi-kilometer trajectories. The system is deployed via an asynchronous FastAPI backend and a responsive dark-mode aerospace telemetry web application featuring live asymmetric power-spline trajectory reconstruction, physical bound clamping, and interactive Runge-Kutta audit verification.*

1. PROBLEM STATEMENT & OBJECTIVES

1.1 Problem Definition

In classical exterior ballistics and atmospheric flight dynamics, a point-mass projectile traveling through the air experiences downward gravitational acceleration and quadratic aerodynamic drag. The governing equations of motion form a coupled non-linear system of second-order differential equations:

$$dx/dt = v_x, \qquad dy/dt = v_y$$
$$dv_x/dt = -(k_{drag} / m) \cdot v \cdot v_x$$
$$dv_y/dt = -g - (k_{drag} / m) \cdot v \cdot v_y$$

where $v = \sqrt{(v_x^2 + v_y^2)}$, $g = 9.81$ m/s², and the lumped aerodynamic drag parameter is defined by $k_{drag} = 0.5 \cdot \rho_0 \cdot C_d \cdot A = 0.003675$ kg/m (with sea-level air density $\rho_0 = 1.225$ kg/m³, sphere drag coefficient $C_d = 0.30$, and cross-sectional area $A = 0.02$ m²).

Because quadratic velocity terms cannot be solved in elementary closed-form analytical integrals, ballistics analysts must rely on iterative step-by-step numerical solvers (e.g., Runge-Kutta RK45, ode45). When thousands of trajectories must be evaluated concurrently —such as in Monte Carlo dispersion analyses, real-time fire-control interception, and autonomous UAV path planning—numerical integration forms an unacceptable computational bottleneck.

1.2 Technical & Aerospace Relevance

- **High-Frequency Interception:** Counter-rocket, artillery, and mortar (C-RAM) interception systems require millisecond decision latency; integration delays directly degrade interception probability.
- **Edge Avionics Constraints:** Lightweight micro-UAVs and guided munitions rely on resource-constrained microcontrollers with limited floating-point units.
- **Digital Twin & Simulation:** Interactive flight simulators require instant recalculation of flight paths as aerodynamic parameters are adjusted by operators.

1.3 Project Objectives

1. **Synthetic Dataset Generation:** Implement event-driven MATLAB (`ode45`) and Python (`scipy.integrate.solve_ivp`) numerical pipelines with ground-impact and apex event detectors across 5,000 distinct flight regimes.
2. **Physics-Informed ML Surrogates:** Formulate custom feature transformations that capture aerodynamic invariants ($v \cdot \sin\theta$, $v \cdot \cos\theta$, $\sin 2\theta$, $\ln m$, $1/m$) to attain an $R^2 \geq 0.99$.
3. **Deterministic Physical Clamping:** Guarantee that ML predictions never violate physical boundaries ($0.05\ m \leq \text{Pred} \leq$ Analytical Vacuum Boundary).
4. **Continuous Spline Path Synthesis:** Develop an asymmetric power spline algorithm that reconstructs continuous flight curves from discrete predictions without numerical steps.
5. **Full-Stack Telemetry Web System:** Deploy an interactive dark-mode telemetry dashboard served via FastAPI and published globally on GitHub Pages.

2. SYSTEM ARCHITECTURE & METHODOLOGY

2.1 Data Flow Architecture

The system follows a clean modular architecture separating the browser user interface, inference service, physics verification, and graphical rendering layers:

```
[User Interface (Telemetry Dashboard)]
   │  Inputs: v0 (20-300 m/s), Angle (5-85°), Mass (0.5-50 kg), Model Selector
   ▼
[FastAPI Backend / Client-Side JS Engine]
   │  1. Expand Features: [v, θ_rad, m, ln(m), 1/m, sin(θ), cos(θ), sin(2θ), vx
   │  2. Execute Trained Regressor Pipeline (Physics Poly / Extra Trees / Decis
   ▼
[Physics Bound Clamping]
   │  Analytical Vacuum Limit: R_vac = v0²·sin(2θ)/g, H_vac = (v0·sinθ)²/(2g)
   │  Enforce: 0.05 m <= Predicted Range <= R_vac
   ▼
[Trajectory Reconstruction & Verification]
   │  Asymmetric Power Spline: Generates 80-point smooth flight curve
   │  Optional: Live Runge-Kutta RK45 Numerical Ground Truth Solve
   ▼
[Telemetry Dashboard Output]
   Hero KPI Cards + Interactive Chart.js Canvas + Holdout Benchmark Table
```

2.2 Dataset Specifications

The training dataset comprises 5,000 systematically gridded trajectories generated via high-precision numerical integration (rtol=10^{-6}, atol=10^{-8}):

- **Velocity (v_0):** 20 to 300 m/s (20 uniform steps)
- **Launch Angle (θ):** 5° to 85° (25 uniform steps)
- **Projectile Mass (m):** 0.5 to 50.0 kg (10 log-spaced steps: 0.5, 0.83, 1.39, 2.32, 3.87, 6.46, 10.77, 17.97, 29.97, 50.0 kg)
- **Target Outputs (5):** Landing Range (m), Peak Altitude (m), Flight Time (s), Apex X coordinate (m), and Impact Speed (m/s).

2.3 Physics-Informed Feature Engineering

Standard regression models trained purely on raw inputs [v, θ, m] fail to capture the exponential and trigonometric relationships of ballistic motion. We engineered a 10-dimensional basis:

$$\Phi(X) = [v,\ \theta_{rad},\ m,\ \ln(m),\ 1/m,\ \sin(\theta),\ \cos(\theta),\ \sin(2\theta),\ v\cdot\cos(\theta),\ v\cdot\sin(\theta)]$$

Aerodynamic Rationale: The term `sin(2θ)` isolates the principal component of vacuum range; `1/m` governs acceleration loss since drag deceleration scales inversely with mass ($a_{drag} = -k\cdot v^2/m$); and `v·cos(θ)` / `v·sin(θ)` represent decoupled orthogonal velocity vectors.

2.4 Project Directory Setup

```
flight-predictor/
├── AGENTS.md                          # Architectural specification & physic
├── requirements.txt                   # Dependencies (FastAPI, scikit-learn,
├── README.md                          # Technical documentation
├── index.html                         # Standalone production web dashboard
├── model_weights.json                 # Serialized client-side model paramet
├── .github/workflows/deploy.yml       # Automated GitHub Actions deployment
├── matlab/generate_flight_data.m      # MATLAB ODE45 event-detection script
├── data/flight_dataset.csv            # 5,000 simulated flight records
├── backend/
│   ├── physics.py                     # Shared aerodynamic constants & featu
│   ├── train_model.py                 # SciPy trajectory generator & model t
│   ├── main.py                        # FastAPI REST server & spline path sy
│   ├── export_client_weights.py       # Model weight extractor for browser e
│   └── models/
│       ├── trajectory_models.joblib   # Serialized trained scikit-learn pipe
│       └── metrics.json               # Validation benchmark metrics
└── frontend/
    ├── index.html                     # Dark-mode aerospace telemetry UI
    └── model_weights.json             # Client-side model weights
```

3. IMPLEMENTATION & CODE SNIPPETS

3.1 Physics Constants & Feature Expansion (`backend/physics.py`)

```
import numpy as np

G = 9.81          # Gravitational acceleration (m/s²)
RHO0 = 1.225      # Sea-level atmospheric density (kg/m³)
CD = 0.30         # Ballistic drag coefficient
A_AREA = 0.02     # Cross-sectional area (m²)
K_DRAG = 0.5 * RHO0 * CD * A_AREA   # Lumped constant = 0.003675 kg/m

def physics_feature_expansion(X: np.ndarray) -> np.ndarray:
    """Transforms raw inputs [velocity, angle_deg, mass] into a 10-D physics ba
    v = X[:, 0]
    ang = X[:, 1]
    m = X[:, 2]
    angle_rad = np.radians(ang)
    return np.column_stack([
        v, angle_rad, m,
        np.log(np.maximum(m, 1e-9)),
        1.0 / np.maximum(m, 1e-9),
        np.sin(angle_rad),
        np.cos(angle_rad),
        np.sin(2.0 * angle_rad),
        v * np.cos(angle_rad),
        v * np.sin(angle_rad),
    ])
```

3.2 Model Pipeline Assembly (`backend/train_model.py`)

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, FunctionTransformer, Polynomi
from sklearn.linear_model import Ridge

physics_poly_pipeline = Pipeline([
    ("physics_expand", FunctionTransformer(physics_feature_expansion, validate=
    ("scaler", StandardScaler()),
    ("poly", PolynomialFeatures(degree=3, include_bias=True)),
    ("ridge", Ridge(alpha=0.01)),
])
```

3.3 Two-Piece Asymmetric Power Spline Reconstruction

Unlike vacuum parabolas which are strictly symmetric about their midpoint, atmospheric drag causes continuous kinetic energy loss, resulting in an apex shifted past 50% of the trajectory and a steep terminal descent angle. The continuous flight profile y(x) is synthesized without running an ODE:

```
def reconstruct_path(R: float, H: float, apex_x: float, n_pts: int = 80):
    apex_x = min(max(apex_x, 0.01), R - 0.01)
```

```
    ratio = apex_x / R
    p_asc = float(np.clip(1.0 / ratio, 1.35, 2.0))
    p_desc = float(np.clip(0.85 / (1.0 - ratio), 2.0, 3.0))

    # Ascent segment (0 -> apex_x)
    xs_asc = np.linspace(0.0, apex_x, n_pts // 2)
    ys_asc = H * (1.0 - ((apex_x - xs_asc) / apex_x) ** p_asc)

    # Descent segment (apex_x -> R)
    xs_desc = np.linspace(apex_x, R, n_pts - n_pts // 2)
    ys_desc = H * (1.0 - ((xs_desc - apex_x) / (R - apex_x)) ** p_desc)

    return np.concatenate([xs_asc, xs_desc[1:]]), np.concatenate([ys_asc, ys_de
```

4. RESULTS & PERFORMANCE EVALUATION

4.1 Holdout Validation Benchmark (20% Split / 1,000 Unseen Trajectories)

The models were trained on 4,000 trajectories and evaluated on 1,000 holdout trajectories under identical random seeds (random_state=42):

Model Key	Algorithm & Pipeline	R^2 Range	R^2 Alt	MAE Range (m)	MAE Alt (m)	Latency (ms)	Speedup vs ODE
physics_poly	Physics Basis + Poly(3) + Ridge	**0.9991**	**0.9995**	**22.29 m**	**7.35 m**	0.266 ms	**3.9x**
extra_trees	ExtraTreesRegressor (50 Estimators)	**0.9992**	**0.9985**	14.36 m	8.55 m	1.851 ms	0.6x
decision_tree	DecisionTreeRegressor (depth=14)	**0.9935**	**0.9865**	47.11 m	32.47 m	**0.056 ms**	**18.6x**
simple_linear	Naive Linear Regression (Baseline)	**0.5358**	**0.5960**	**510.98 m**	**232.66 m**	0.049 ms	21.2x

4.2 Analysis & Discussion

1. **Non-Linear Underfitting of Linear Models:** The naive `simple_linear` baseline failed with an R^2 of ~0.54 and an average range error exceeding 510 meters. This quantitatively confirms that quadratic velocity damping cannot be modeled linearly.
2. **Physics-Informed Performance:** The proposed `physics_poly` model reached an R^2 of 0.9991, reducing range error to 22.3 m across distances spanning over 6,000 meters, while maintaining an execution speed of 0.266 ms (3.9x faster than numerical ODE).
3. **Decision Tree Latency:** The single decision tree achieved 0.056 ms inference latency (18.6x faster than numerical ODE), making it ideal for micro-avionics applications.

4.3 Telemetry Dashboard Verification

The web application was launched locally on port 8000 and verified using headless Chrome DevTools Protocol (CDP). Below is the captured telemetry output under the **High Drag Regime** (v_0 = 250 m/s, θ = 45°, m = 1.5 kg):



Figure 4.1: Live Aerospace Telemetry Dashboard showing ML Predicted Drag Curve, Ideal Vacuum Parabola, and Real-Time RK45 ODE Ground Truth Verification.

Verification Readout: Under this high-drag regime, the theoretical vacuum range is 6,371.1 m. Atmospheric drag dissipates 88.1% of the projectile's forward kinetic energy, reducing actual landing distance to 732.3 m. The ML model predicted 759.4 m (Δ = 27.1 m, 96.4% accuracy) in only 1.6 ms.

5. CONCLUSION & FUTURE SCOPE

5.1 Conclusion

The project successfully designed, implemented, and validated a physics-informed Machine Learning surrogate for ballistic flight dynamics with quadratic aerodynamic drag. By transforming raw kinematic parameters into an aerodynamic basis, the `physics_poly` model delivers near-perfect accuracy ($R^2 > 0.999$) and sub-millisecond inference speeds. The full system is deployed both as a local FastAPI inference service and as a standalone client-side application accessible globally via GitHub Pages.

5.2 Future Scope

- **6-DOF Aerodynamic Dynamics:** Extending point-mass equations to six degrees of freedom to incorporate projectile spin, Magnus force, and crosswind gust fields.
- **Barometric Density Stratification:** Modeling altitude-dependent atmospheric decay ($\rho(y) = \rho_0 e^{-y/H}$) for high-altitude ballistic trajectories.
- **Edge Firmware Compilation:** Compiling the polynomial weights into static C headers for deployment on ARM Cortex-M microcontrollers and STM32 drone autopilots.
- **Physics-Informed Neural Networks (PINNs):** Training deep surrogate networks using governing ODE residuals directly in the loss function.

6. REFERENCES

1. **Nelson, R. C.** (1998). *Flight Stability and Automatic Control* (2nd ed.). McGraw-Hill Education.
2. **McCoy, R. L.** (2012). *Modern Exterior Ballistics: The Launch and Flight Dynamics of Symmetric Projectiles*. Schiffer Publishing.
3. **Pedregosa, F., et al.** (2011). "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research (JMLR)*, 12, 2825–2830.
4. **Virtanen, P., et al.** (2020). "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python." *Nature Methods*, 17(3), 261–272.
5. **Karniadakis, G. E., et al.** (2021). "Physics-informed machine learning." *Nature Reviews Physics*, 3(6), 422–440.
6. **Tipler, P. A., & Mosca, G.** (2007). *Physics for Scientists and Engineers* (6th ed.). W. H. Freeman.
7. **FastAPI Documentation**. (2026). *FastAPI: High performance Python web framework*. Available: https://fastapi.tiangolo.com

8. **Live Project Repository**. (2026). *Instant Flight Trajectory Predictor*. Available: https://github.com/vivekcv08-jpg/flight-trajectory-predictor